

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**

Department of Computer Science and Engineering

**ASSESSMENT TOOL 2 – CASE STUDY**

|                         |                                                                                                          |                      |                                   |
|-------------------------|----------------------------------------------------------------------------------------------------------|----------------------|-----------------------------------|
| <b>Course Code:</b>     | CSA06                                                                                                    | <b>Course Title:</b> | Design and Analysis of Algorithms |
| <b>CO Assessed:</b>     | <b>CO3:</b> Articulation (BL4, BL5)                                                                      | <b>Assessment:</b>   | Assessment Tool 2 – Case Study    |
| <b>Weightage:</b>       | 20% of CIA                                                                                               | <b>Total Marks:</b>  | 100                               |
| <b>CO3 Statement:</b>   | Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication |                      |                                   |
| <b>Student Name:</b>    | B.Shyam raj                                                                                              | <b>Reg. No.:</b>     | 192421176                         |
| <b>Section / Batch:</b> | 2024                                                                                                     | <b>Date:</b>         |                                   |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

| <b>Question Type</b> | <b>Focus Area</b>                                                                                              | <b>Questions</b> |
|----------------------|----------------------------------------------------------------------------------------------------------------|------------------|
| Case Study           | Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication | Q1               |

**SECTION A – CASE STUDY****Code of Conduct**

I B.Shyam raj(192421176) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

## RUBRICS FOR CALCULATION

| Case Study              |           |                                                                  |                                                |                                         |                                                      |
|-------------------------|-----------|------------------------------------------------------------------|------------------------------------------------|-----------------------------------------|------------------------------------------------------|
| Criteria                | Weightage | Level 4 – Exemplary                                              | Level 3 – Proficient                           | Level 2 – Developing                    | Level 1 – Beginning                                  |
| Understanding of Case   | 25%       | Thorough understanding; clearly identifies all key issues        | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly |
| Application of Concepts | 25%       | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps   | Limited application of concepts         | Fails to apply relevant concepts                     |
| Analysis                | 20%       | In-depth analysis with strong reasoning and insights             | Good analysis with logical reasoning           | Basic analysis with limited depth       | Weak or no analysis; lacks reasoning                 |
| Solution Design         | 20%       | Innovative, feasible solutions with strong justification         | Logical solutions with adequate justification  | Basic solutions with weak justification | Incorrect or no solution provided                    |
| Clarity                 | 10%       | Clear, well structured, and easy to understand                   | Organized and understandable presentation      | Limited clarity and structure           | Poor clarity; difficult to understand                |

### Marks Evaluation:

| Criteria              | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|-----------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Understanding of Case | 25% |                     |                      |                      |                     |

|                           |     |  |  |  |  |
|---------------------------|-----|--|--|--|--|
| Application of Concepts   | 25% |  |  |  |  |
| Analysis                  | 20% |  |  |  |  |
| Solution Design           | 20% |  |  |  |  |
| Clarity                   | 10% |  |  |  |  |
| <b>Total Marks (100):</b> |     |  |  |  |  |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name:             | Name:              | Name:              |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

## QUESTIONS

Q1

Merge Sort vs Quick Sort  
100 marks

1/1 answered

## Q1 Merge Sort vs Quick Sort

100 marks

Evaluate the robustness of Merge Sort and Quick Sort under noisy or partially corrupted datasets. Introduce controlled data inconsistencies and observe behavior. Record execution time and output correctness. Interpret results and justify algorithm reliability in imperfect conditions.

## Your Code

```
1 import random
2 import time
3 def merge_sort(arr):
4     if len(arr) <= 1:
5         return arr
6     mid = len(arr) // 2
7     left = merge_sort(arr[:mid])
8     right = merge_sort(arr[mid:])
9     return merge(left, right)
10 def merge(left, right):
11     result = []
12     i = j = 0
13     while i < len(left) and j < len(right):
14         if left[i] <= right[j]:
15             result.append(left[i])
16             i += 1
17         else:
18             result.append(right[j])
19             j += 1
20     result += left[i:]
```

## Input

stdin input

## Output

```
Original noisy data: [810, 787,
26, 952, 'bad_data', 251,
'bad_data', 200, 'bad_data', 386,
845, 'bad_data', 110, 258, 277,
728, 577, 81, 644, 651]
```

```
21     result += right[j:]
22     return result
23 def quick_sort(arr):
24     if len(arr) <= 1:
25         return arr
26     pivot = arr[len(arr) // 2]
27     left = [x for x in arr if x < pivot]
28     mid = [x for x in arr if x == pivot]
29     right = [x for x in arr if x > pivot]
30     return quick_sort(left) + mid + quick_sort(right)
31 def add_noise(data, noise_level=0.1):
32     data = data.copy()
33     n = int(len(data) * noise_level)
34     for _ in range(n):
35         idx = random.randint(0, len(data) - 1)
36         data[idx] = "bad_data"
37     return data
38 def clean(data):
39     return [x for x in data if isinstance(x, int)]
40 data = [random.randint(1, 1000) for _ in range(20)]
41 noisy_data = add_noise(data, 0.2)
42 print("Original noisy data:", noisy_data)
43 cleaned_data = clean(noisy_data)
44 start = time.time()
45 merged = merge_sort(cleaned_data)
46 end = time.time()
47 print("\nMerge Sort Result:", merged)
48 print("Merge Sort Time:", end - start)
49 print("Correctly sorted:", merged == sorted(merged))
50 start = time.time()
51 quicked = quick_sort(cleaned_data)
52 end = time.time()
53 print("\nQuick Sort Result:", quicked)
54 print("Quick Sort Time:", end - start)
55 print("Correctly sorted:", quicked == sorted(quickd))
```